Patrick Foulis
CS 455 Computer Networks
Lab 5
22 September 2026

I. Sockets and States

```
tcp4       0      0  *.52993                *.*
LISTEN

tcp4       0      0  10.128.5.170.52991     142.251.111.188.5228
ESTABLISHED

tcp4       0      0  10.128.5.170.52990     142.251.111.188.5228
ESTABLISHED
```

4-tuple:

- Source IP (Local IP): 10.128.5.170
- Source Port (Local Port): 52990
- Destination IP (Remote IP): 142.251.111.188
- Destination Port (Remote Port): 5228

II. Two Connections, One port

```
tcp4      0     0  10.128.5.170.53330     150.171.110.24.443     ESTABLISHED
tcp4      0     0  10.128.5.170.53329     96.16.195.233.443      ESTABLISHED
```

The operating system uses the local port (e.g., 53330 vs 53329) to tell the two conversations apart, assigning a unique random port number to each outgoing connection while keeping the source IP and destination port the same.

III. UDP to a Closed Port

```
MacBook-Air-4 ~ % printf 'anybody there?\n' | nc -u -w1 127.0.0.1 9099
; echo "exit status: $?"
exit status: 0

MacBook-Air-4 ~ % nc -u -l 9098
one datagram, no connection
```

This proves that UDP sends and forgets, there is no confirmation that the receiver received the message from the sender. The first exited with a pass even though it was never received, the pass measures that it was sent from the OS. UDP will not provide any confirmation that it was received if an application requires it.

IV. TCP to the same closed port

```
nc -v -z 127.0.0.1 9099 ; echo "exit status: $?"
nc: connectx to 127.0.0.1 port 9099 (tcp) failed: Connection refused
exit status: 1
printf 'anybody there?\n' | nc -u -w1 127.0.0.1 9099 ; echo "exit
status: $?"
exit status: 0
```

This proves that TCP has verification of receiving the message as it came back with a fail when it tried to connect to the given IP address. When UDP doesn't check with any verification, because it does not have any reason too. TCP will resend or in this case notify if it doesn't receive it.

V. The TCP Header

- Source Port: 57644
- Destination Port: 9099
- Sequence Number: 1510356782
- Acknowledgment Number: 0 (Because this is the selection of the SYN packet)
- Data Offset: 44 bytes
- Flags
  - SYN set
- Window: 65535
- Calculated Window: 65535
- Checksum: 0xfe34
- Urgent Pointer: 0
- Options:
  - TCP Option – Maximum segment size: 16344 bytes
  - TCP Option – Window scale: 6 (multiply by 64)
  - TCP Option – Timestamps: TSval 2377632800, TSecr 0
  - TCP Option – SACK permitted: True (or supported)
  - TCP Option – No-Operation (NOP): Padding bytes used to align option boundaries
  - TCP Option – End of Option List (EOL): Marks the end of the TCP options header

VI. Initial Sequence Numbers

Client Initial Sequence Number: 1510356782
Server Initial Sequence Number: 3673742609

If zero was picked for the initial sequence number, any traffic that was delayed from a previous transmission would interfere with the one coming in as the sequence numbers would always be relatively close to each other. Using a random and large number not only prevents this problem from occurring, it also prevents TCP hijacking because the sequence and where it starts is unpredictable. If it started at zero, an attacker knows where to begin hijacking from in the sequence every time.

VII. Opening

- Packet 1
  - Flags: SYN
  - Sequence Number: 1510356782
  - Acknowledgment Number: N/A
- Packet 2
  - Flags: SYN, ACK
  - Sequence Number: 3673742609
  - Acknowledgment Number: 1510356783
- Packet 6
  - Flags: ACK
  - Sequence Number: 1510356805
  - Acknowledgment Number: 3673742610

The SYN flag consumes one sequence number to synchronize the connection. Thus the acknowledgement number would be 1 + the SYN flag's number, even though there was no information sent.

VIII. Closing

My .pcap file combined the closing of the connection from 4 packets to two.

Packet 6: Client to Server
- 127.0.0.1:57644 -> 127.0.0.1:9099
- Flags: [FIN, ACK]

Packet 9: Server to Client
- 127.0.0.1:9099 ->127.0.0.1:57644
- Flags: [FIN, ACK]

TCP connections are full-duplex. Thus each end of the two way connection needs to be closed. Meaning 4 packets need to be sent to do so (2 FIN and 2 ACK) . The opening combines the step 2 and 3 of the closing into one packet. Because no-one was listening on the port. The kernel sent the RST,ACK flag to tell netcat to stop the attempt at completing the three way handshake.

IX. The Refusal

RST Packet Details

- Direction: Server to Client (127.0.0.1:9099 -> 127.0.0.1:57655)
- Flags: [RST, ACK]

Because no-one was listening on the port. The kernel sent the RST,ACK flag to tell netcat to stop the attempt at completing the three way handshake. During UDP there was no flag sent that no one was listening. This is because UDP does not look for acknowledgment that the packet was received. It does not need to be told to not retry the handshake because it just won't.

X. Time-Wait

tcp4 0 0 127.0.0.1.56679 127.0.0.1.9099 TIME_WAIT

The reason that this TIME_WAIT command is there is to ensure that the final ACK command is actually received. Unlike UDP, TCP needs confirmation that the packet arrived. So TIME_WAIT is there if that ACK is lost in transit, the peer will retransmit its FIN, which can only be properly re-acknowledged if the socket remains in TIME-WAIT. Waiting for the maximum life of the Segment times 2, prevents delayed packets from old transmissions from being improperly dealt with.

XI. Two Transports

UDP Packet Count: 2 (1,2)
TCP Packet Count: 10 (3,…,12)
A-Record Returned: 208.80.154.224

TCP Breakdown:

- Packets 3, 4, 5 (Three-Way Handshake):
  - Client sends SYN (Packet 3), Server replies SYN, ACK (Packet 4), Client confirms ACK (Packet 5).
- Packets 6, 7, 8, 9 (DNS Query and Data Exchange):
  - Client sends DNS Query over TCP (Packet 6).
  - Server sends transport ACK (Packet 7).
  - Server responds with DNS Answer 208.80.154.224 (Packet 8).
  - Client sends transport ACK (Packet 9).
- Packets 10, 11, 12 (Connection Teardown):

- Client initiates close with FIN, ACK (Packet 10).
- Server sends FIN, ACK (Packet 11).
- Client acknowledges with ACK (Packet 12).

DNS defaults to UDP because it only takes two packets for a standard request. This avoids the latency that may happen if TCP was used and the extra packets were needed for acknowledgment and tear down of the connection. TCP is only used when there is a reason that guaranteed reliability or segment ordering is necessary.

XII. Kernel State

** I AM ON MAC OS AND DO NOT HAVE A WINDOWS OR LINUX MACHINE TO RUN THIS ON**

```
patrickfoulis@Patricks-MacBook-Air-4 ~ % curl -s -o /dev/null 'https://speed.cloudflare.com/__down?bytes=50000000' &
sleep 2 ; netstat -an -p tcp | grep '\.443' | head -20
wait
[1] 52335
[1]  + done       curl -s -o /dev/null 'https://speed.cloudflare.com/__down?bytes=50000000'
tcp4       0      0  10.128.5.170.56739     44.215.132.93.443      CLOSE_WAIT
tcp4       0      0  10.128.5.170.56736     23.203.187.234.443     ESTABLISHED
tcp4       0      0  10.128.5.170.56727     162.247.243.29.443     ESTABLISHED
tcp4       0      0  10.128.5.170.56724     44.199.180.130.443     ESTABLISHED
tcp4       0      0  10.128.5.170.56738     172.66.0.218.443       TIME_WAIT
tcp4       0      0  10.128.5.170.56740     162.159.140.220.443    TIME_WAIT
tcp4       0      0  10.128.5.170.58287     151.101.35.6.443       ESTABLISHED
tcp4       0      0  10.128.5.170.58286     54.162.235.133.443     ESTABLISHED
tcp4       0      0  10.128.5.170.58285     54.165.7.254.443       ESTABLISHED
tcp4       0      0  10.128.5.170.58270     34.233.49.149.443      ESTABLISHED
tcp4       0      0  10.128.5.170.58217     3.217.166.173.443      ESTABLISHED
```

This was the output I was able to get
Gemeni Provided me this sample Linux output as I am not able to run it natively.

```
ESTAB      0      0    10.128.5.170:58217   172.66.0.218:443
         cubic rtt:18.421/2.115 mss:1460 rto:204 cubic cwnd:10 ssthresh:8
```

Identified Fields:

- Congestion Control Algorithm: cubic (Linux default algorithm)
- Smoothed Round-Trip Time (SRTT): 18.421 ms
- Congestion Window (cwnd): 10 segments
- RTT: 18.421
- Retransmission Timeout (RTO): 204 ms

RTT is the average time for a round trip for the ACK to travel. RTO takes RTT and adds a buffer which is equal to RTO = SRTT + 4 * RTTVAR. Thus it would make sense that RTT is smaller than RTO. If this was reversed, then any normal variation in the network time round trip would cause the window to close before valid ACK packets get there. Thus there would be a lot of packets resent that weren't actually lost, causing bandwidth to be used up that wasn't necessary.

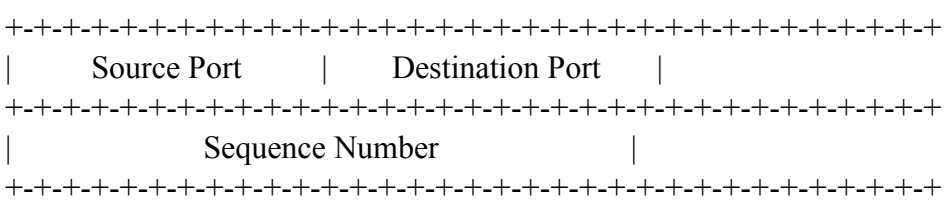
XIII. Window scale and bandwidth-delay product

Wscale = 6 (client)
Wscale = 7 (server)
BDP = 230.25 KB

An unscaled 64KB window is not enough to keep a 100 Mbit link busy over an 18.42\ms path by a factor of roughly 3.5x. Without any window scaling the sender would spend a lot of time idling when waiting for ACK packets. Window scaling allows the window size to be extended past 64KB allowing TCP to fully utilize modern high-bandwidth, high-latency network paths.

XIV. Answers to the six specification questions

1)



Row 1: 16 bits (Source Port) + 16 bits (Destination Port) = 32 bits (4 bytes)
Row 2: 32 bits (Sequence Number) = 32 bits (4 bytes)
Total: 8 bytes (64 bits total)

The first two rows account for **8 bytes**. Comparing this to Section 4, these 8 bytes make up the first two 32-bit words of the total TCP header size (which has a 20-byte minimum base header)

2)
"If SYN is present the sequence number is the initial sequence number (ISN) and the first data octet is ISN+1."

Because the SYN bit is set during connection establishment, the value contained in the Sequence Number field represents the randomly chosen start by the sender, rather than pointing directly to payload data. The SYN flag itself consumes 1 bit so the first byte of the payload is ISN + 1

3)

"Data Offset: 4 bits. The number of 32 bit words in the TCP Header."
The field is 4 bits wide so the maximum int it can hold is 15. Since the unit is 32-bit words (4 bytes per word), 15 times 4 bytes = 60 bytes

4)
Order of eight control bits (left-to-right in diagram):
I. CWR (Congestion Window Reduced)
II. ECE (ECN-Echo)
III. URG (Urgent Pointer field significant)
IV. ACK (Acknowledgment field significant)
V. PSH (Push Function)
VI. RST (Reset the connection)
VII. SYN (Synchronize sequence numbers)
VIII. FIN (No more data from sender)

CWR and ECE were recently added to support Explicit Congestion Notification (ECN). ECE allows a receiver to notify the sender that the packet was received. The CWR is a flag that is used to notify that it reduced its congestion window to mitigate network congestion in acknowledgment of the ECE flag.

5)
Fields and Lengths:
- Source Port: 16 bits (2 bytes)
- Destination Port: 16 bits (2 bytes)
- Length: 16 bits (2 bytes)
- Checksum: 16 bits (2 bytes)
- Total Header Size: 8 bytes (64 bits)

TCP's minimum header size with none of the options used is 20 bits. Whereas UDP's is 8 bits, which is 12 smaller than TCP.

6)
"*This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed*."

UDP runs with a fire and forget mindset. There is no bits or bandwidth wasted waiting for an acknowledgment that it was received or any intention of resending. This means that if the packet is never received then it is just lost. There is no system in place to recover it and make sure that

the information arrived to its destination. Hence why in Section 2 it appeared to have sent the packet even though there was no one listening on the other end.

XV. Reflection

I would miss the fact that TCP resends and ensures that information is received before it closes the connection. Imagine you were to open a website and there were missing photos, the fonts were all different, and there were sections of the text that was missing. This is what would happen if we did not have the assurance of reception that TCP provides when we go to access a website over the network.